

PIP 2026

Game Design Document

Game Feel

- Fun
- exploration
- accomplishment
- puzzling

Inspirations

- Rogue
- Text-based adventure games
- every REST API ever

Rules/ Principles

- Don't question the input
- Do not look back
- Click the button

Gaming Genre

- RPG

Thematic and style Genre

- Text
- Window
- Font
- most powerful GPU in the world, depending on user

Backstory

You may choose what you are, but the world will learn on who that truly is through your actions alone.

Explore the vastness of your imagination, if you think, you have what it takes.

Write into input fields, Press buttons, Read, not necessarily in that order.

Role-playing game

Required Subsystems

- Dialogue (no choices)
- Movement and Interaction
- Inventory (no management)
- Menu Screens (main, pause, save/load, options)

Required Infrastructure

- Menu System
- To-Database Serialization
- Game System

Gameplay Loop/s

- Start: Navigate through menu
- Spiel: exploration and dialogue
- Ende: dialogue shows the way to the goal/friends we make along the way

Overview

"placeholder" will be primarily REST API to some RPG. Players are able to safely connect and save their progress. The Backend handles all relevant operations to connection and storage, while the Unity Frontend provides Users with the necessary interface and feedback of API requests.

Users explore a fictional world, interact with NPCs via dialogue.

We aim for a robust system that can be expanded upon, depending on use cases.

- Unity Frontend
- English
- Software architecture

Project Management

Participants use GitHub for Project and repository relevant uses. Project Backlog to document Issues and progress.

We use Proton for protected file sharing. Each participant curates their personal backlog in the proton sheet.

Formatting document uploaded to GitHub.

GDD is updated according to progress of issues.

Components

Component Software

- Unity (C#)
- Node.js (js)
- XAMPP / MySQL (SQL)

Frontend

- UI Assets
- Input
- Menu Windows
 - Main
 - Pause
 - 🎮 Options
 - 🎮 save/load
- Game
 - World
 - Dialogue
 - NPC/ user
 - Skip button
 - 🎮 Combat

Backend

- REST API with CRUD operations
 - /session (🔑 id,started_at, ended_at)
 - /input (🔑 session_id,times_a_pressed,tiles_moved,etc)
 - /time (🔑 session_id,time_spent_in_menus,time_spent_exploring)
 - /npc (🔑 session_id, 🔑 npc_id,times_talked_to,lines_of_dialogue_skipped)
 - 🎮 /user (🔑 id,name,created_at)
 - /settings
 - /audio (🔑 user_id,music_volume,sfx_volume)
 - /video (🔑 user_id,screen_tint)
 - 🎮 /generation
 - /item
 - /weapon
 - /room
- Server-Client Connection
- Input Validation / Statement Preparation
- 🎮 Encryption

Database

- Persistence

Backend

REST API using CRUD operation. Aiming for stateless design, aside maybe for access token.

Basic Structure

The client makes calls to the API, which generates responses.



RESTful Specifications

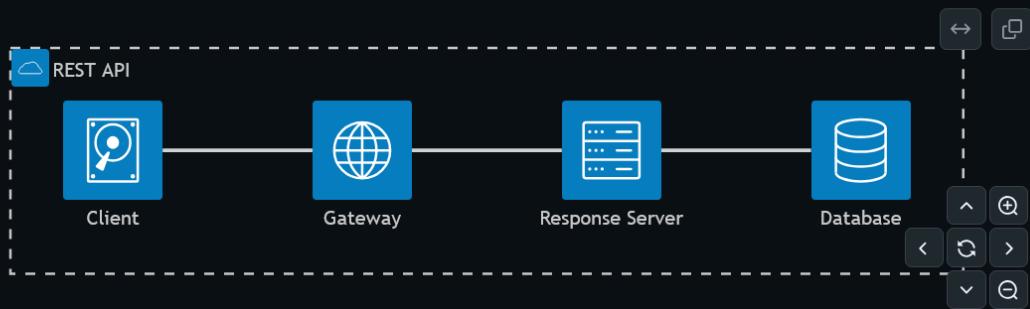
As a RESTful API, the endpoints follow a uniform and predictable pattern.

- `/statistics/session`
- `/statistics/input`
- `/statistics/interaction/npc`
- `/statistics/time`

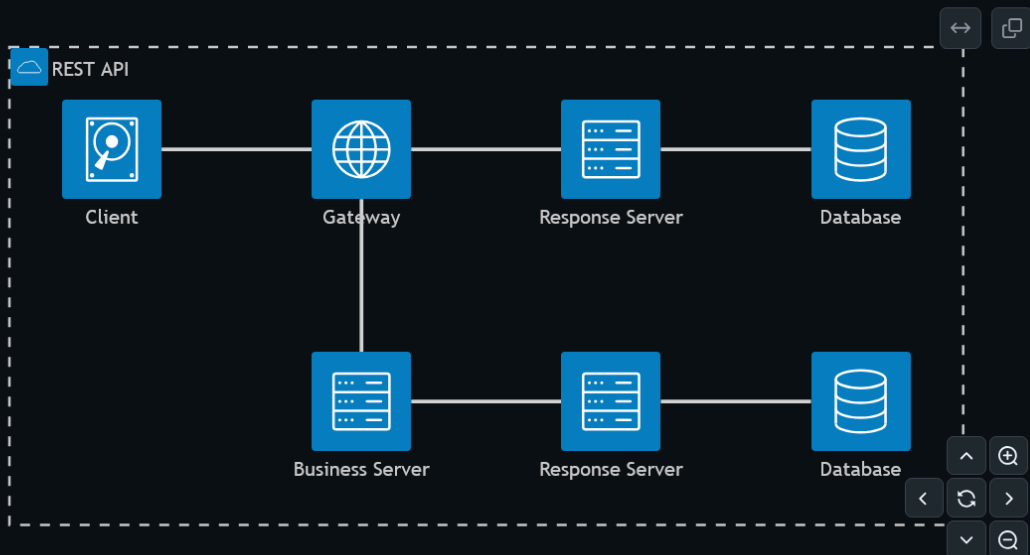
Servers implementing the API should be able to seamlessly communicate with each other and maintain a clear and traceable chain of communication between each node.



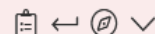
Following these principles, the api is made arbitrarily extensible, with servers able to specialize and handle specific tasks, such as security, routing, or data retrieval.



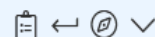
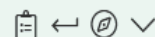
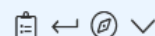
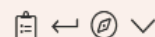
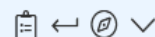
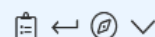
Example of a branched architecture where the API is accessed via a gateway server.



admins Secured Admin-only calls

**DELETE****/session/{sessionId}** deletes a session entry

developers Operations available to regular developers

**GET****/session** retrieves session data**POST****/session** adds a session entry**GET****/session/{sessionId}** retrieves specific session data**PUT****/session/{sessionId}** updates a session entry**GET****/session/{sessionId}/input** retrieves input data**GET****/session/{sessionId}/time** retrieves time data

The API handles Input validation, Client information processing, their session, save and load profiles/games. Throughout the session it collects relevant data for statistical purposes, which are game relevant only.

Basic API Structure in Code

```
41
42
43 #region Unity Editor
    1 reference | ☒ Unity Serialized Field
44 | | [SerializeField] private string apiHost = "127.0.0.1" ;
    1 reference | ☒ Unity Serialized Field
45 | | [SerializeField] private string apiPort = "4141" ;
46
47 | | [Header("Endpoints")]
    1 reference | ☒ Unity Serialized Field
48 | | [SerializeField] private ApiPath session ;
    1 reference | ☒ Unity Serialized Field
49 | | [SerializeField] private ApiPath input ;
    1 reference | ☒ Unity Serialized Field
50 | | [SerializeField] private ApiPath interaction ;
    1 reference | ☒ Unity Serialized Field
51 | | [SerializeField] private ApiPath time ;
52 #endregion
53
54
55 #region Static Properties
    8 references | ☒ Unity Serialized Field
56 | | public static Endpoints Endpoints = new() ;
    1 reference
57 | | public static string Host => _instance.apiHost ;
    1 reference
58 | | public static string Port => _instance.apiPort ;
59 #endregion
60
61
```

Basic Response Data Structure

```
7 // GetSessionResponse
  Maria Lindling, yesterday | 1 author (Maria Lindling)
8 [Serializable]
  2 references
9 ▼ internal class GetSessionResponse : RequestResponse
10 {
11     1 reference
12     public SessionRowData[] data;
13     0 references
14     public ResponseContext<SessionContextKeys> context;
15 }
16 // PostSessionData
17 Maria Lindling, yesterday | 1 author (Maria Lindling)
18 [Serializable]
19 1 reference
20 ▼ internal class PostOrPutSessionData
21 {
22     1 reference
23     public string started_at ;
24     1 reference
25     public string ended_at ;
26 }
27 // PostSessionResponse
28 Maria Lindling, yesterday | 1 author (Maria Lindling)
29 [Serializable]
30 4 references
31 ▼ internal class PostOrPutSessionResponse : RequestResponse
32 {
33     2 references
34     public int insert_id ;
35     0 references
36     public ResponseContext<SessionContextKeys> context;
37 }
```

API Paths example, used to pass on Webrequests

```

19  public virtual IEnumerator Post( int[] ids, byte[] data, Action<string> onResult )
20  {
21      StringBuilder uri = new (BaseURI) ;
22
23      foreach( int id in ids ) { if(id == -1) break ; uri.Append('/') ; uri.Append(id) ; }
24
25      yield return StartCoroutine( PostRequest( uri.ToString(), data, onResult ) );
26  }
27  3 references
28  public virtual IEnumerator Post( int[] ids, byte[] data ) => Post( ids, data, (s) => {} ) ;
29  0 references
30  public virtual IEnumerator Post( int id, byte[] data, Action<string> onResult ) => Post( new[] {id}, data, onResult ) ;
31  2 references
32  public virtual IEnumerator Post( int id, byte[] data ) => Post( new[] {id}, data, (s) => {} ) ;
33  2 references
34  public virtual IEnumerator Post( byte[] data, Action<string> onResult ) => Post( new[] {-1}, data, onResult ) ;
35  3 references
36  public virtual IEnumerator Post( byte[] data ) => Post( new[] {-1}, data, (s) => {} ) ;
37  #endregion
38  Maria Lindling, 6 days ago * refactored the RestApi class and split request ...
39
40  #region PUT
41  3 references
42  public virtual IEnumerator Put( int[] ids, byte[] data, Action<string> onResult )
43  {
44      StringBuilder uri = new (BaseURI) ;
45
46      foreach( int id in ids ) { if(id == -1) break ; uri.Append('/') ; uri.Append(id) ; }
47
48      yield return StartCoroutine( PutRequest( uri.ToString(), data, onResult ) );
49  }
50  1 reference
51  public virtual IEnumerator Put( int[] ids, byte[] data ) => Put( ids, data, (s) => {} ) ;

```

```

17
18  #region GET
19  3 references
20  public override IEnumerator Get( int[] ids ) => throw new InvalidOperationException( "SessionPath does not handle more than one id." ) ;
21  2 references
22  public override IEnumerator Get( int id ) { throw new NotImplementedException() ; }
23  1 reference
24  public override IEnumerator Get() => Get(-1) ;
25  #endregion
26
27  #region POST
28  1 reference
29  public override IEnumerator Post( int id, byte[] data ) { throw new NotImplementedException() ; }
30  #endregion
31
32  #region PUT
33  1 reference
34  public override IEnumerator Put( int id, byte[] data ) { throw new NotImplementedException() ; }
35  #endregion
36
37  #region DELETE
38  1 reference
39  public override IEnumerator Delete( int id ) { throw new NotImplementedException() ; }
40  #endregion
41  }

```

```

10  {
11
12  1 reference | © Unity Script (1 asset reference) | Maria Lindling, yesterday | 1 author (Maria Lindling)
13  public class SessionPath : ApiPath
14  {
15  4 references
16  internal Dictionary<int,SessionRowData> Cache { get ; } = new () ;
17  #endregion Maria Lindling, 6 days ago * add shell classes for api paths; add restapi pr...
18
19  #region GET
20  3 references
21  public override IEnumerator Get( int[] ids ) => throw new InvalidOperationException( "SessionPath GET does not handle more than
22  2 references
23  public override IEnumerator Get( int id )
24  {
25      yield return StartCoroutine( base.Get( id, (text) => {
26          GetSessionResponse res = JsonUtility.FromJson<GetSessionResponse>(text) ;
27
28          int entriesAdded = 0 ;
29          int entriesUpdated = 0 ;
30          foreach( SessionRowData data in res.data )
31          {
32              if( Cache.TryGetValue( data.id, out SessionRowData cachedData ) )
33              {
34                  if( true /* && data.recorded_at > cachedData.recorded_at */ )

```

No problems have been detected in the workspace.

Database

Persistent JSON files on Clients and their activities. MySQL, controlled via XAMPP.

(Insert structure)

(Insert example of client)

Unity structs that are translated to JSON by the SaveManager and passed on to the API

```
28 public struct UserProfileData
29 {
30     0 references
    public int userId ;    // Either clientId or SessionId to identify correspo
    0 references
31     public string userName ;    // Neat to know
    0 references
32     public UserSaveStatistics statistics ;
    0 references
33     public UserProfileConfiguration config ;
    0 references
34     public UserSaveData SaveData ;
35 }
36
37 /// <summary>
38 /// The Settings hold Interface information, e.g. Volume Settings
39 /// </summary>
    1 reference | You, 4 days ago | 1 author (You)
40 public struct UserProfileConfiguration
41 {
42     0 references
    public float MasterVolume;
    0 references
43     public float MusicVolume;
    0 references
44     public float SfxVolume;
    0 references
45     public bool Muted;
46 }
47
48 /// <summary>
49 ///   DataPackage to transfer statistics between Application and Database
```

Basic Response Handler

```

    response.status(400) ;
    response.json( responseBody ) ;
    return ;
  }
  /** End */

  /** Query SQL Server */
  let [ rows, fields ] = await sql_connection.execute( sql, sql_params ) ;
  /**
   * TODO: error handling
   * ERR: rows contains no elements
   */
  if( rows.length == 0 )
  {
    responseBody.ok = false ;
    responseBody.error = "No matching data found.";
    response.json( responseBody ) ;
    return ;
  }
  responseBody.data = {
    "id":          rows[0][fields[0].name],
    "started_at":  rows[0][fields[1].name],
    "ended_at":    rows[0][fields[2].name],
    "recorded_at": rows[0][fields[3].name],
  } ;
  /** End */

  response.json( responseBody ) ;
} ) ) ;

app.get( "/statistics/input", asyncMiddleware( async (request,response,next) => {
  /***/
  let responseBody = responseBodyGET ;
  #!/usr/bin/node      Maria Lindling, last week + add session GET, add session POST (shell) ...
  "use strict" ;
  export { responseBodyPOST } ;

  const responseBodyPOST = {
    "ok": true,
    "cacheable" : true,
    "custody_chain" : [],
    "insert_id": null,
    "context": null,
    "_meta": "[usage/handling information for client]",
    "error": ""
  }
}

```

Frontend

World Map, controlled by controls that need to be determined yet.

Other windows are:

Menu

Provides basic user interface to navigate either into game or other menu windows.



Inventory

Representing player inventory, either as list or in compartments, undecided yet.

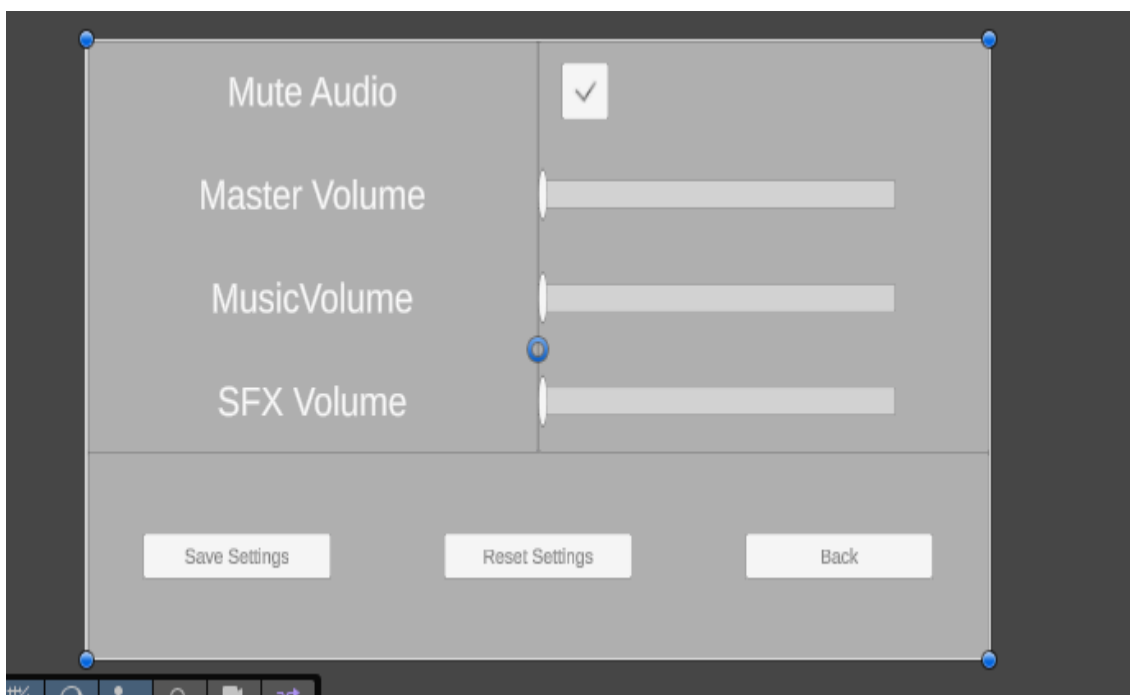
Pause

In-game menu to navigate to different parts of the game. E.g. to get back to the main menu, toggle the options window, open the save/load window, etc. The Main Menu is currently the Pause Menu as well.

Options

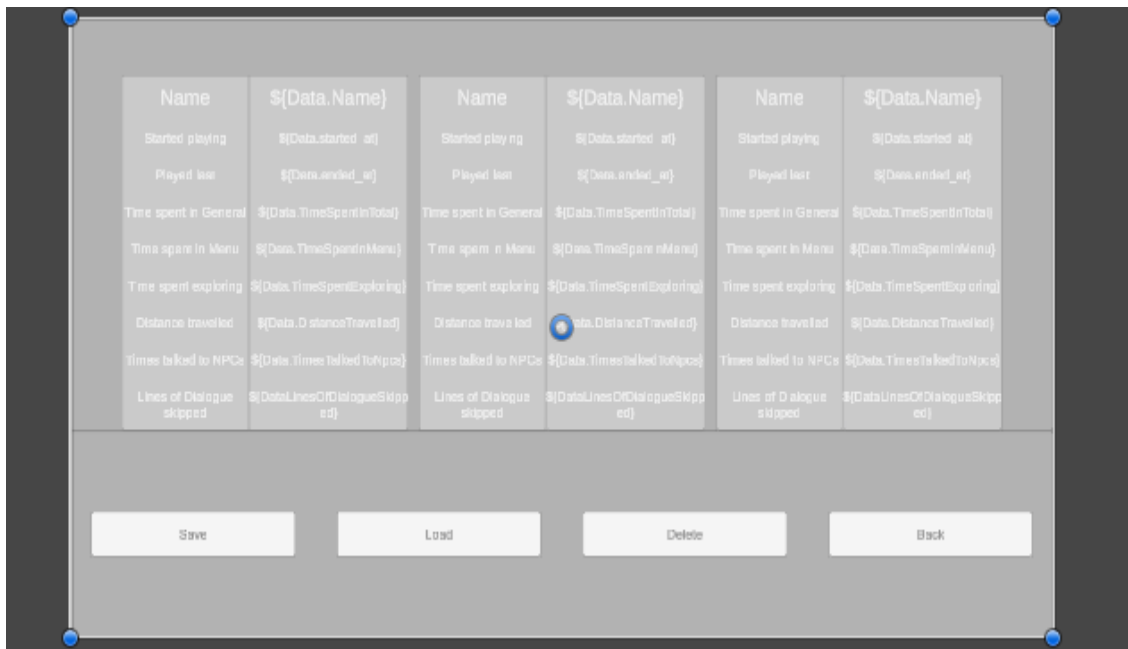
Provide users with settings that are automatically updated to their profile.

Settings are saved in the Profile Configuration of the User

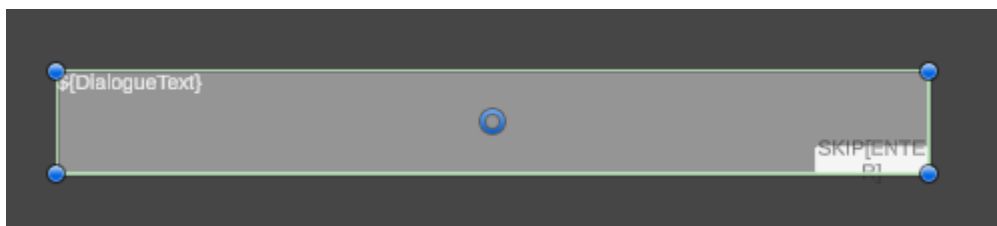


Save/Load

Serving the user as interface to save and load profiles and/or game saves.



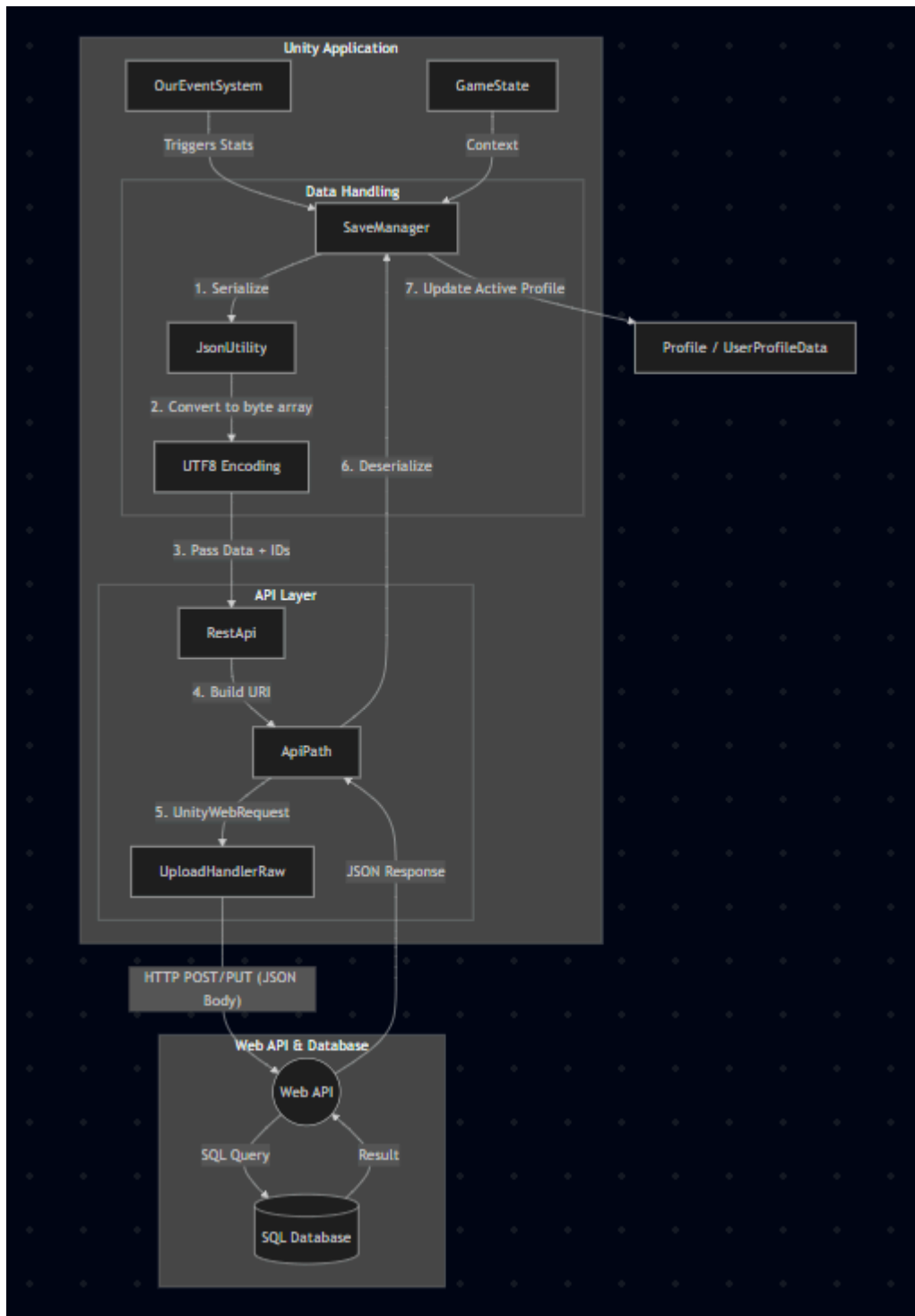
Dialogue



All of them are in a single UI Canvas and represent their own Panel.

SaveManager

Application Manager that serves as translator between Unity and Database. Requests to call on CRUD operation through the API to safely store Data of the users



Example of SaveManager methods